



Analyse et Programmation Orientée Obj

Présentation 7

Héritage, redéfinition, chaîne de constructeurs,
polymorphisme et lien dynamique



Haute École Léonard de **Vinci**

SCIENCES ET TECHNIQUES

Héritage



Héritage

Mécanisme permettant de définir une nouvelle classe à partir d'une classe existante

La nouvelle classe hérite des attributs et méthodes de la classe existante. Elle peut définir des attributs et des méthodes propres

Quand l'utiliser ?

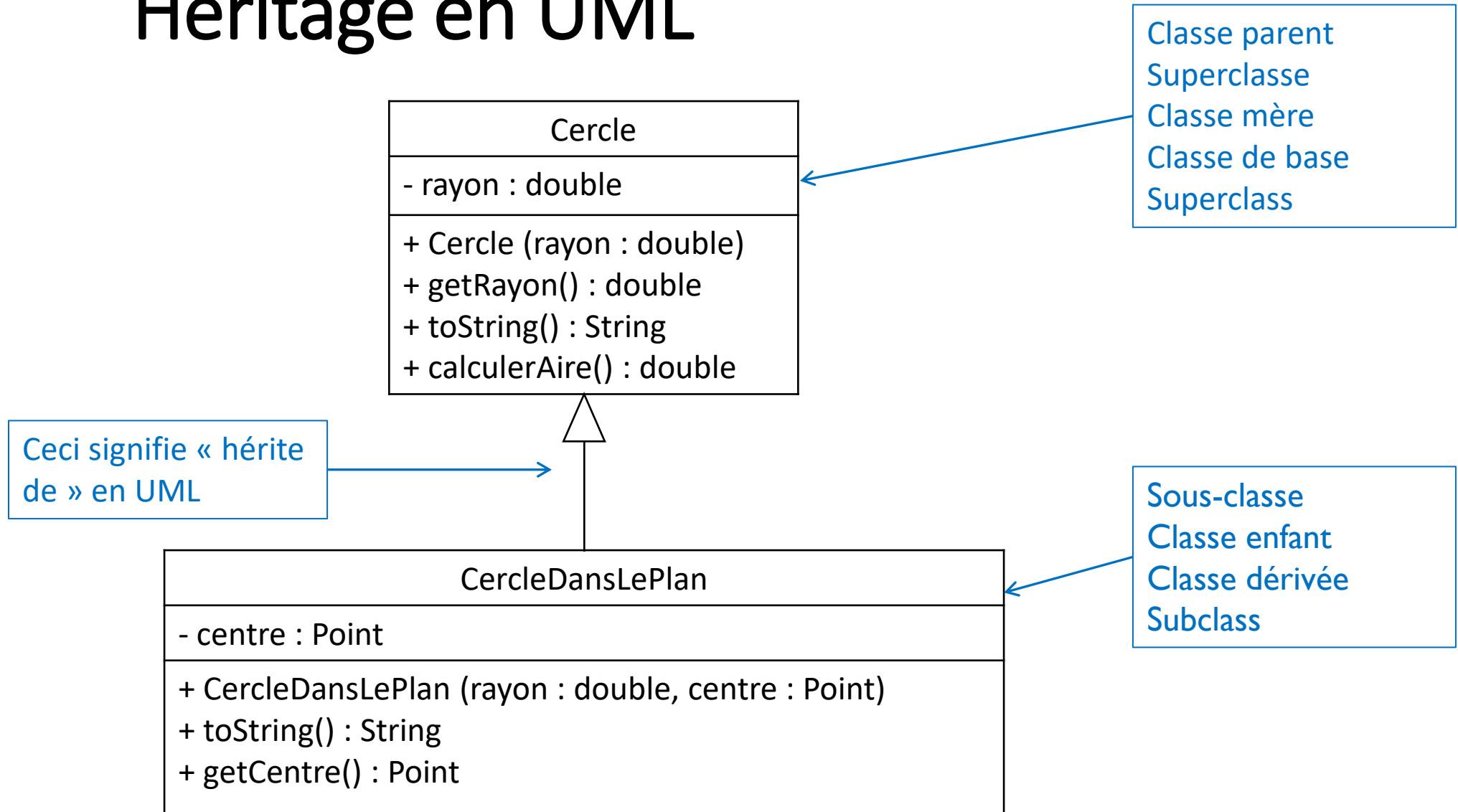
- Quand une classe possède tous les attributs et méthodes d'une autre

Pourquoi l'utiliser ?

- Pour éviter d'écrire deux fois les mêmes attributs et méthodes ; pour éviter de dupliquer du code



Héritage en UML





Héritage en Java

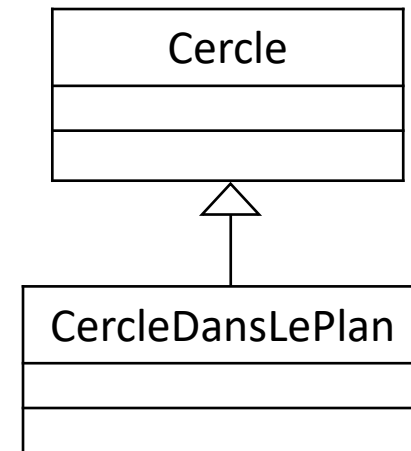
```
public class Cercle {  
    private double rayon;  
  
    public Cercle(double rayon) {  
        this.rayon = rayon  
    }  
  
    public double calculerAire() {  
        return rayon * rayon * Math.PI;  
    }  
    ...  
}
```

En Java, l'absence d'**extends** est équivalent à **extends Object**

En Java, on place le mot « **extends** » pour signifier qu'une classe hérite d'une autre classe !

Le constructeur de la classe parent est invoqué par le constructeur de la classe qui hérite
= 1^{ère} instruction du constructeur !

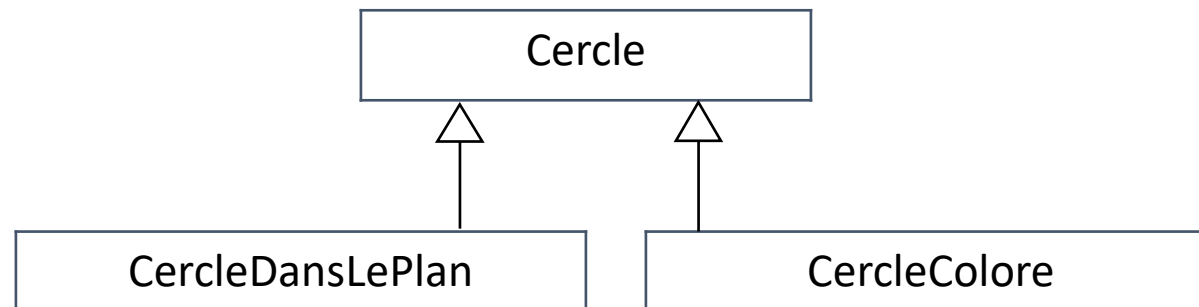
```
public class CercleDansLePlan extends Cercle {  
    private Point centre;  
  
    public CercleDansLePlan(double rayon, Point centre) {  
        super(rayon) ;  
        this.centre = centre;  
    }  
  
    public Point getCentre() { ... }  
    ...  
}
```



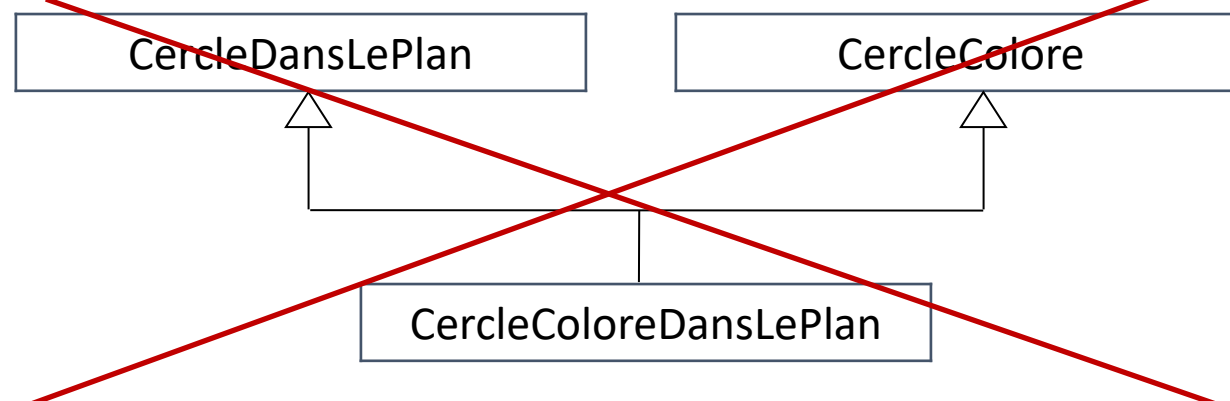


Plusieurs héritières mais un parent unique

- Plusieurs classes peuvent hériter d'une même classe

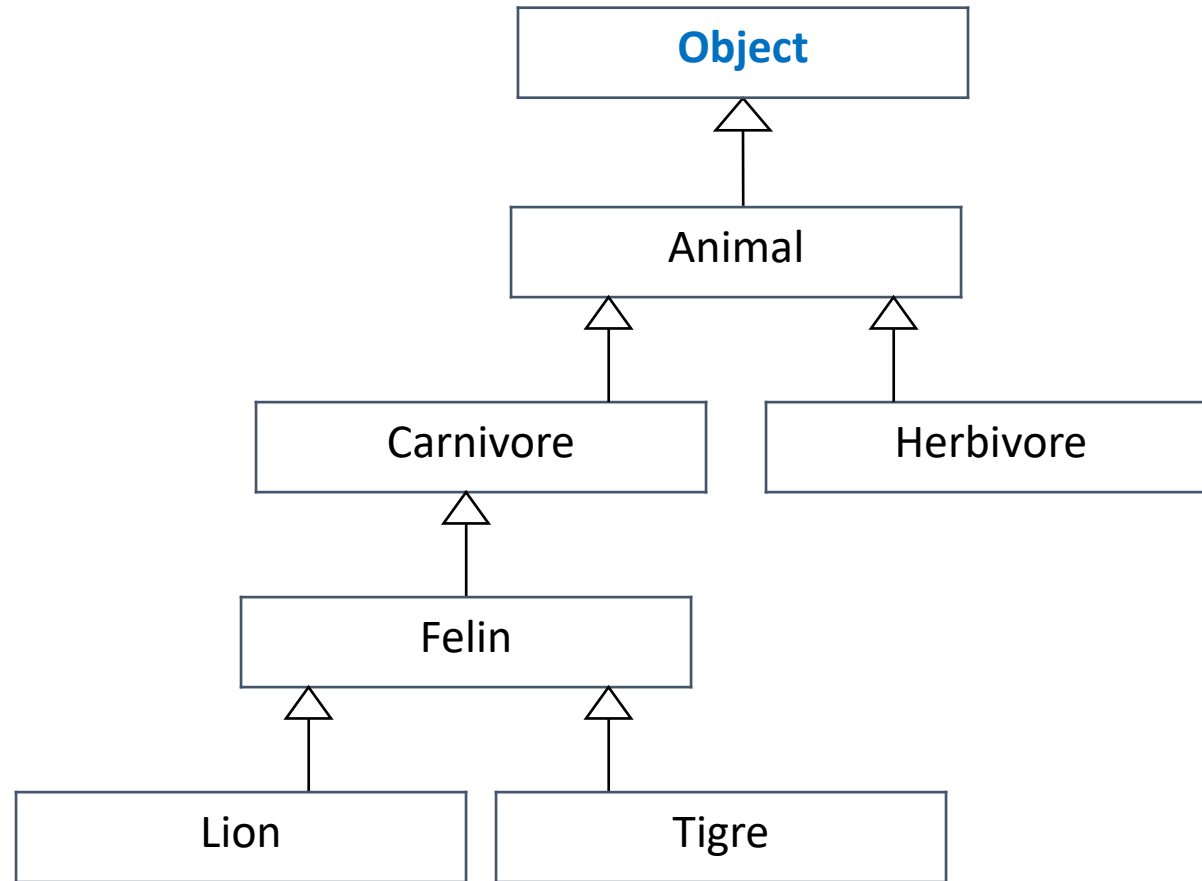


- Mais en Java, une classe ne peut pas hériter de plusieurs classes





Niveaux d'héritage





Héritage des attributs

- Héritage de tous les **attributs** de la classe parent
 - Ex : un objet de la classe CercleDansLePlan a un rayon (rayon = attribut de la classe Cercle)
- Pas accès direct aux attributs **privés** de la classe parent

```
public class Cercle {  
    private double rayon;  
  
    ...  
}
```

```
public class CercleDansLePlan extends Cercle{  
    ...  
    public double setRayon(double rayon){  
        this.rayon = rayon;  
    }  
    ...  
}
```

Erreur de compilation





Héritage des méthodes

- Héritage des **méthodes visibles**
(pas les méthodes privées)
de la classe parent

```
public class Cercle {  
    private double rayon;  
  
    public Cercle(double rayon) {  
        this.rayon = rayon  
    }  
  
    public double calculerAire() {  
        return rayon * rayon * Math.PI;  
    }  
    ...  
}
```



```
...  
Point point = new Point(2,3);  
CercleDansLePlan cercleP = new CercleDansLePlan(1,point);  
System.out.println("Aire du cercle : " + cercleP.calculerAire());  
...
```



Ce code est-il correct ? Pourquoi ?

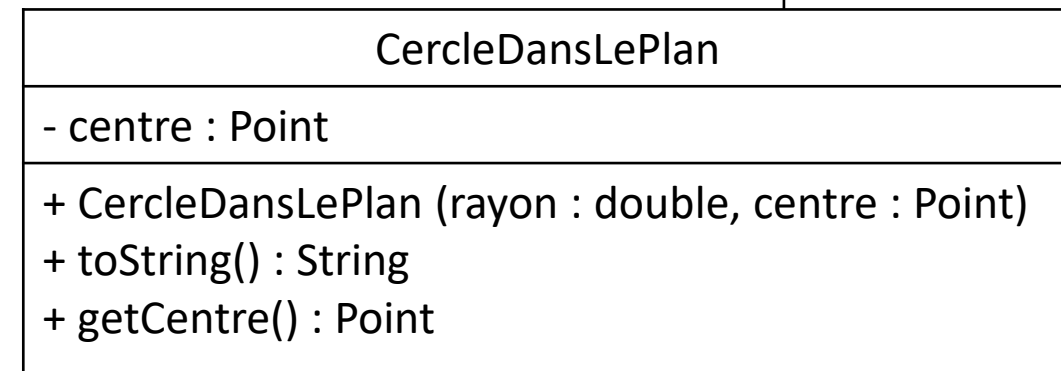
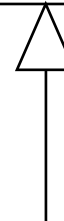
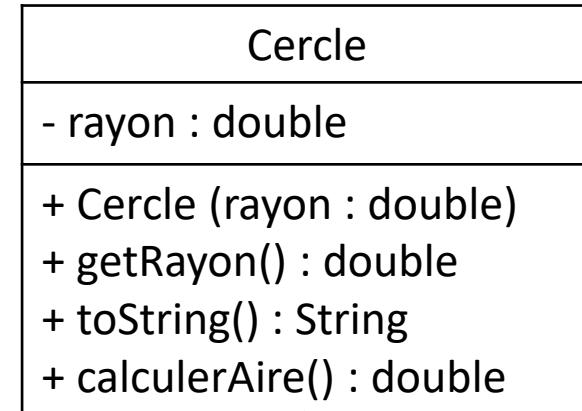
```
public class Cercle {  
    private double rayon;  
    public Cercle(double rayon){ this.rayon = rayon; }  
    ...  
}
```

```
public class CercleDansLePlan extends Cercle {  
    private Point centre;  
    public CercleDansLePlan(double rayon, Point centre) {  
        super(rayon);  
        this.centre = centre;  
    }  
    ...  
    public String toString() {  
        return "Cercle de rayon : " + rayon + " de centre " + centre;  
    }  
}
```



Que va afficher ce programme ?

```
public class TestHeritage {  
    public static void main(String[] args) {  
        Point p = new Point(1,3);  
        CercleDansLePlan cp = new CercleDansLePlan(4, p);  
        System.out.println(cp.calculerAire());  
    }  
}
```





Redéfinition



Redéfinir une méthode

Il faut parfois réécrire une méthode de la classe parent dans la sous-classe afin qu'elle se comporte différemment

C'est ce qui est appelé **overriding** ou **redéfinition**

Une méthode ne peut être redéfinie que si elle est

- visible par la classe enfant
- pas static

Pour redéfinir une méthode dans une sous-classe il faut

- la même signature (= nom de la méthode + type des paramètres)
- que le type de retour soit le même ou une sous-classe du type renvoyé



Exemple de redéfinition

```
public class Cercle{  
    private double rayon;  
    ...  
    public String toString(){  
        return "Cercle de rayon " + rayon;  
    }  
    ...  
}
```

Redéfinition de la
méthode `toString()`
de la classe `Cercle`

```
public class CercleDansLePlan extends Cercle{  
    private Point centre;  
    ...  
    public String toString() ← {  
        return super.toString() + " centré en " + centre;  
    }  
    ...  
}
```

Ceci fait appel à la méthode `toString()` qui est définie dans la superclasse `Cercle`.
Syntaxe pour appeler une méthode de la classe parent : **`super.nomDeLaMéthode(...)`**



Exemple de redéfinition

```
public class TestHeritage {  
    public static void main(String[] args) {  
        Point point = new Point(2,3);  
        Cercle c1 = new Cercle(2);  
        CercleDansLePlan c2 = new CercleDansLePlan(1,point);  
        System.out.println(c1.toString());  
        System.out.println(c2.toString());  
    }  
}
```

Que va afficher ce programme ?

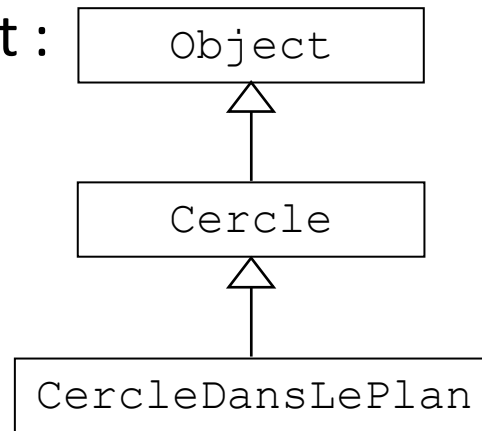


Chaîne de constructeurs



Héritage et constructeurs

- **Pas** d'héritage des **constructeurs**
- Définition dans la sous-classe de ses propres constructeurs
- En java, **tout constructeur invoque un constructeur de la classe parent**
- Chaîne d'héritage > Object :



En UML on ne note pas cette classe Object



Invoquer le constructeur de la classe parent



```
public class Cercle{  
    private double rayon;  
  
    public Cercle(double rayon){  
        // super() ;  
        this.rayon = rayon  
    }  
    ...  
}
```

Appel au constructeur de la classe Object fait de façon implicite (on ne l'écrit pas !)

Appel au constructeur de la classe parente fait explicitement : toujours **première** instruction du constructeur !

```
public class CercleDansLePlan extends Cercle{  
    private Point centre;  
  
    public CercleDansLePlan(double rayon, Point centre){  
        super(rayon) ;  
        this.centre = centre;  
    }  
    ...  
}
```



Invoquer le constructeur de la classe parent

```
public class Cercle{  
    private double rayon;  
  
    public Cercle(double rayon){  
        this.rayon = rayon  
    }  
  
    public Cercle(){  
        this(1.0);  
    }  
    ...  
}
```

Il faut écrire le constructeur sans paramètres car il sera invoqué par défaut



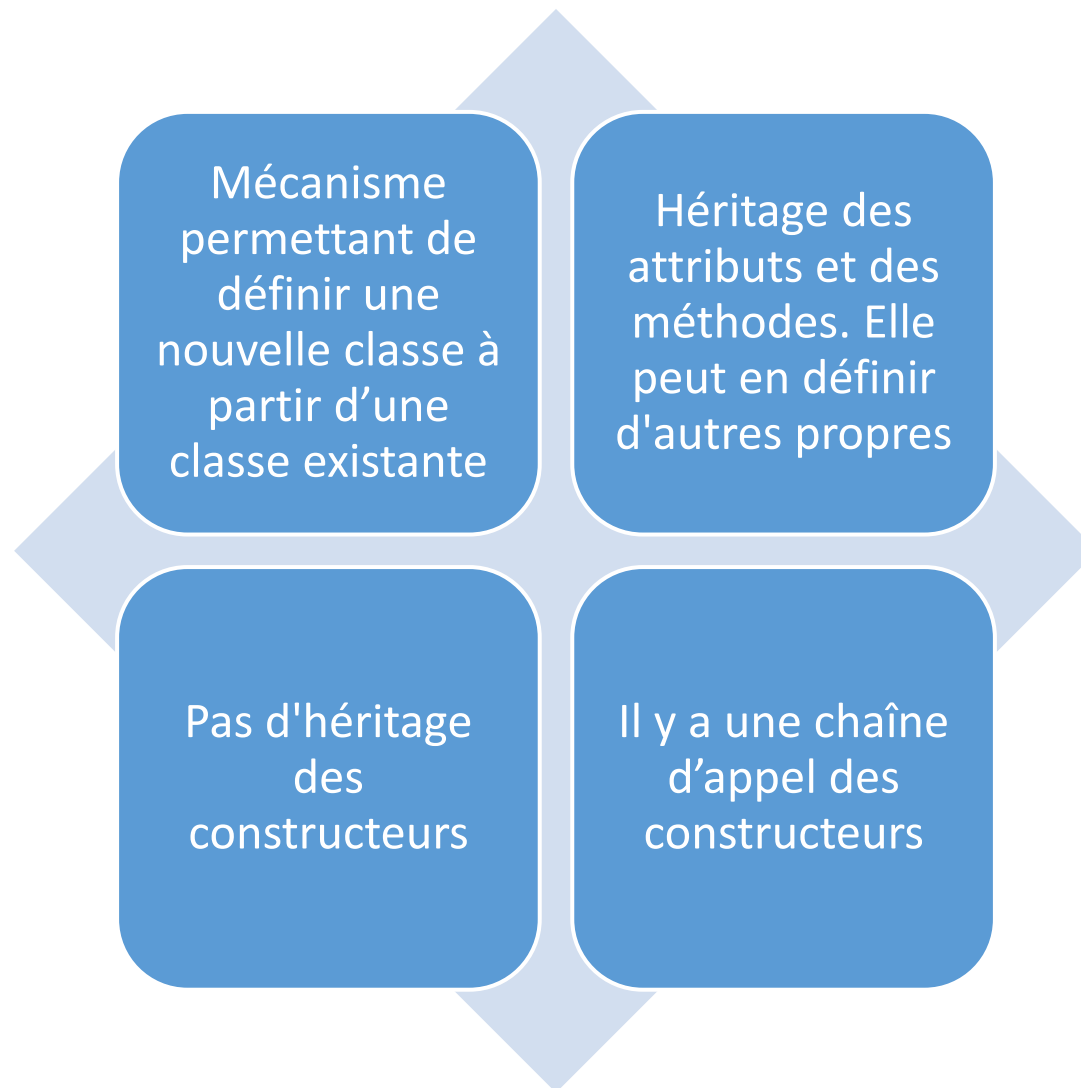
! Erreur de compilation !



```
public class CercleDansLePlan extends Cercle{  
    private Point centre;  
  
    public CercleDansLePlan(Point centre){  
        super();  
        this.centre = centre;  
    }  
    ...  
}
```



Héritage : récapitulatif



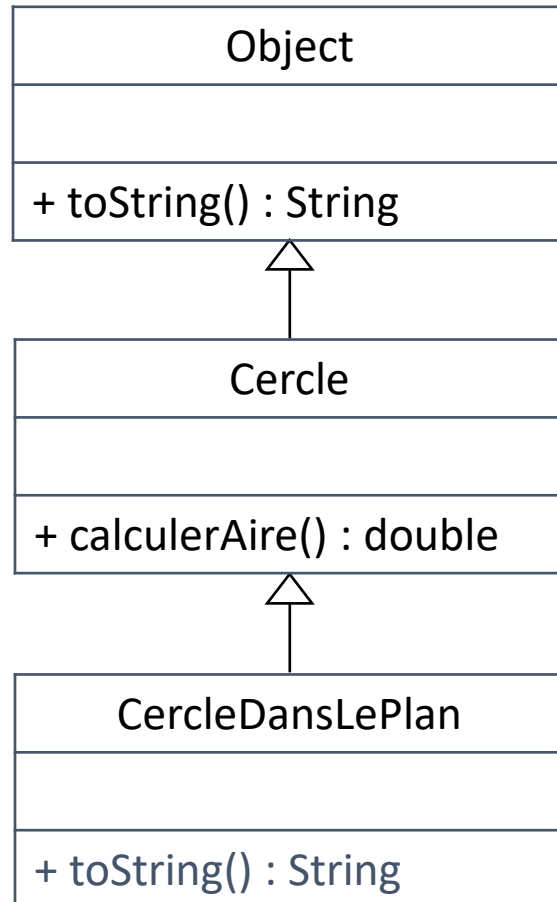


Polymorphisme

*Le **polymorphisme**, du grec πολύμορφος / polúmorphos (« multiforme »), caractérise la capacité à se présenter sous différentes formes*



Polymorphisme



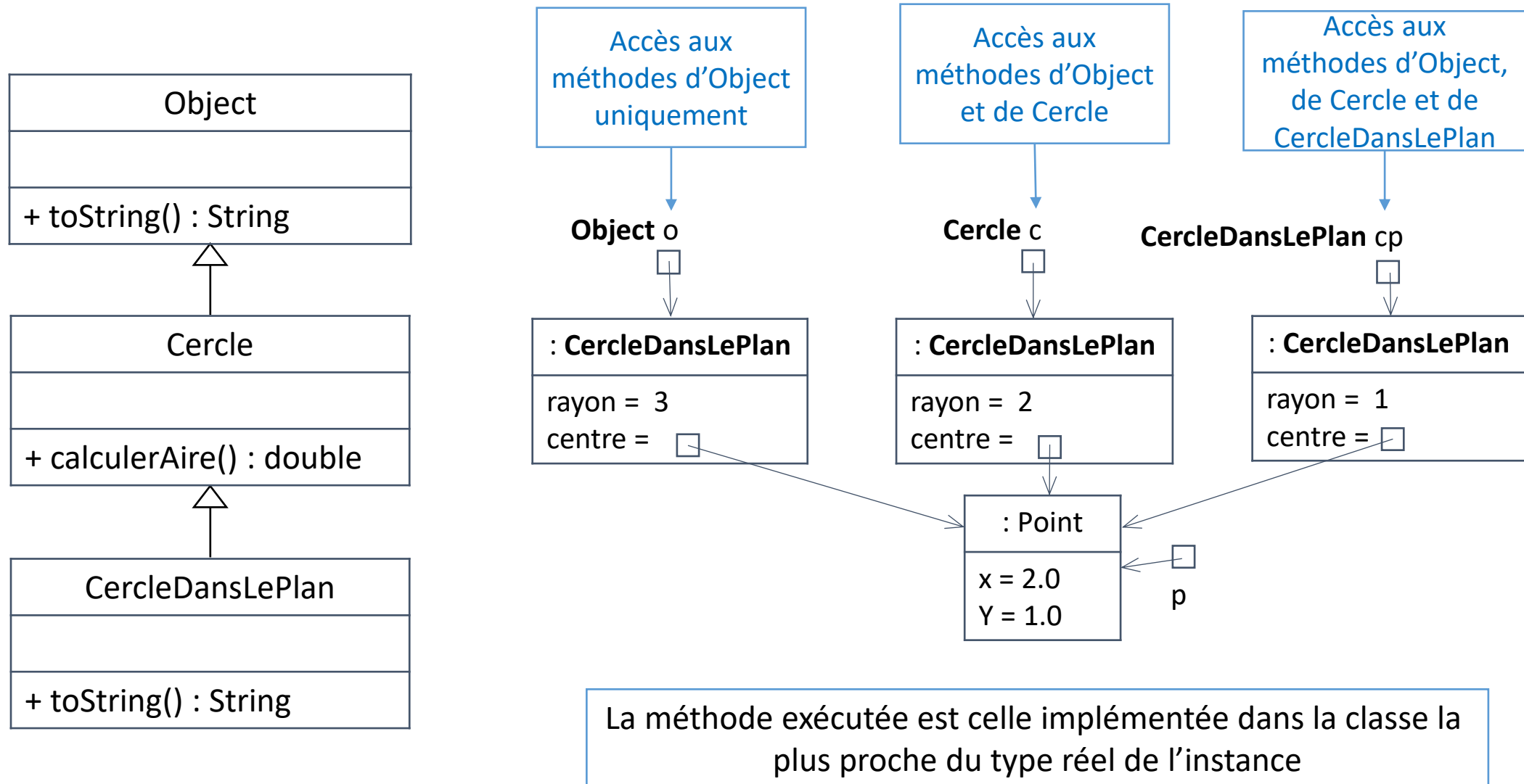
Une instance de `CercleDansLePlan` sera assignable à n'importe quel type de sa chaîne d'héritage

```
Point p = new Point(2,1);
Object o = new CercleDansLePlan(3,p);
Cercle c = new CercleDansLePlan(2,p);
CercleDansLePlan cp = new CercleDansLePlan(1,p);
```

Une instance peut donc prendre **plusieurs formes** (types).

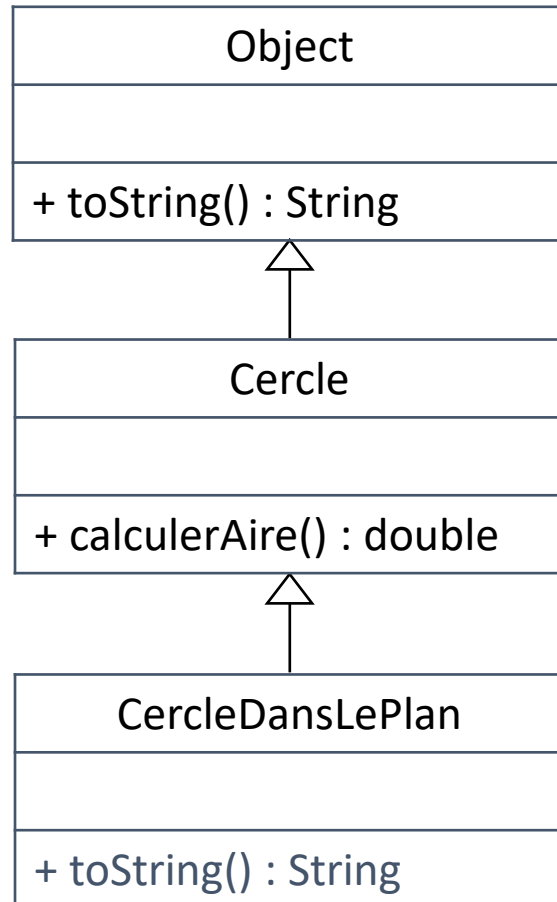


Polymorphisme





Changement de type

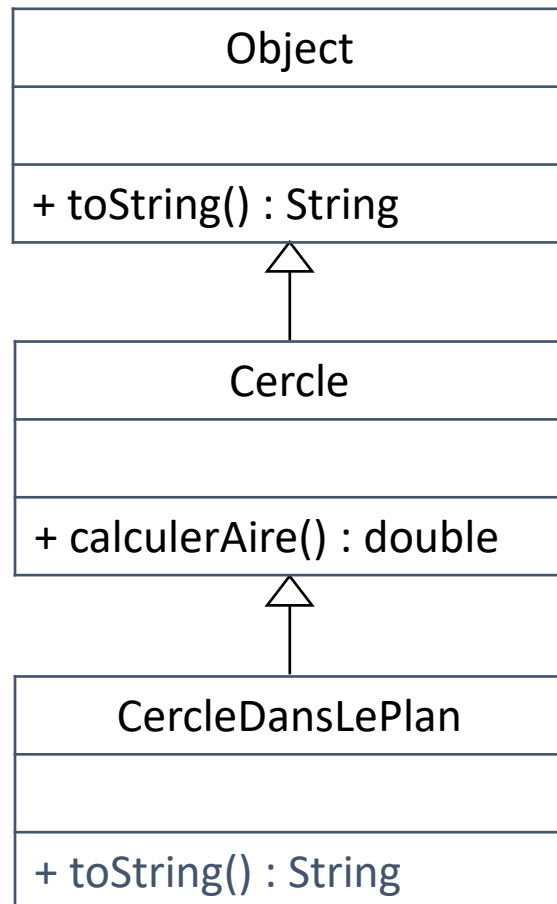


- Une instance peut être assignée à des variables de plusieurs types (classe)
- **Seuls les attributs/méthodes** définis par le type de la **variable** sont **utilisables**
- On peut assigner une variable d'un certain type vers un type du même niveau ou **plus haut** dans la chaîne d'héritage.

```
Point p = new Point(2,1);
CercleDansLePlan cp = new CercleDansLePlan(1,p);
Cercle c = cp;
Object o1 = c;
Object o2 = cp;
```




Transtypage (cast)



- Pour assigner vers un type plus bas dans la chaîne d'héritage, il faut « caster »
 - Opération dynamique qui peut échouer à l'exécution
 - Lance une **ClassCastException** en cas d'échec

```
Cercle c1 = new Cercle(3);
Point p = new Point(2,1);
Object o = new CercleDansLePlan(1,p);
Cercle c2 = (Cercle)o;
CercleDansLePlan cp1 = (CercleDansLePlan)o;
CercleDansLePlan cp2 = (CercleDansLePlan)c2;
CercleDansLePlan cp3 = (CercleDansLePlan)p;
CercleDansLePlan cp4 = (CercleDansLePlan)c1;
```



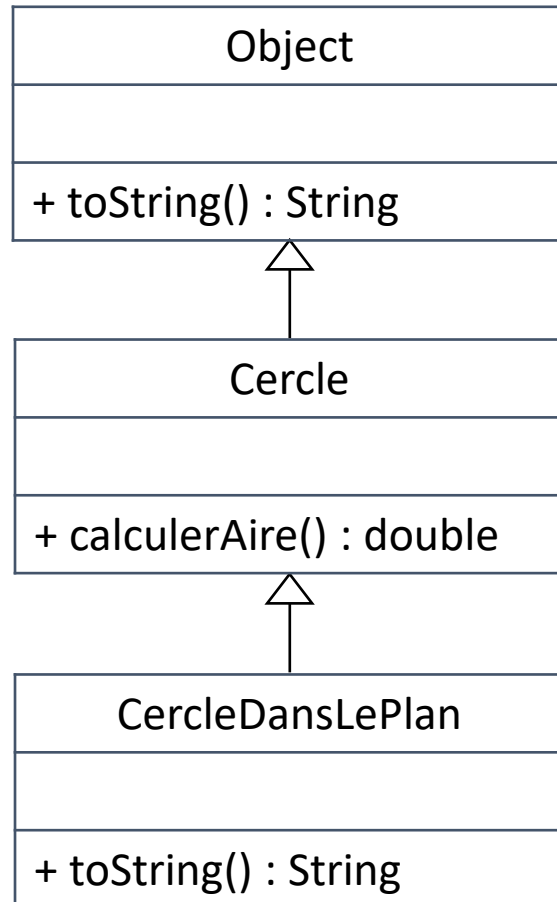
Lien dynamique

Une instance d'une certaine classe X peut être assignée à des variables de plusieurs types

- Le **type de la variable** décide quels sont les **attributs/méthodes disponibles**
- Cependant lors d'un appel de méthode disponible de par le type déclaré, c'est bien la **méthode implémentée dans la classe X qui sera appelée**
 - Si X redéfinit cette méthode, c'est cette implémentation qui sera exécutée.
 - Sinon c'est Java qui cherche une implémentation dans la classe parent, puis le parent du parent et ainsi de suite jusqu'à Object



Lien dynamique : exemple



```
Point p = new Point(2,1);
```

```
Object o = new CercleDansLePlan(1,p);
```

```
o.toString();
```

```
o.calculerAire();
```

✓ toString() existe dans Object. Utilisation de toString() le plus proche de CercleDansLePlan, c-à-d toString() de la classe CercleDansLePlan

✗ calculerAire() n'existe pas dans Object. Pas d'accès à la méthode donc

```
Cercle c = new CercleDansLePlan(5,p);
```

```
c.toString();
```

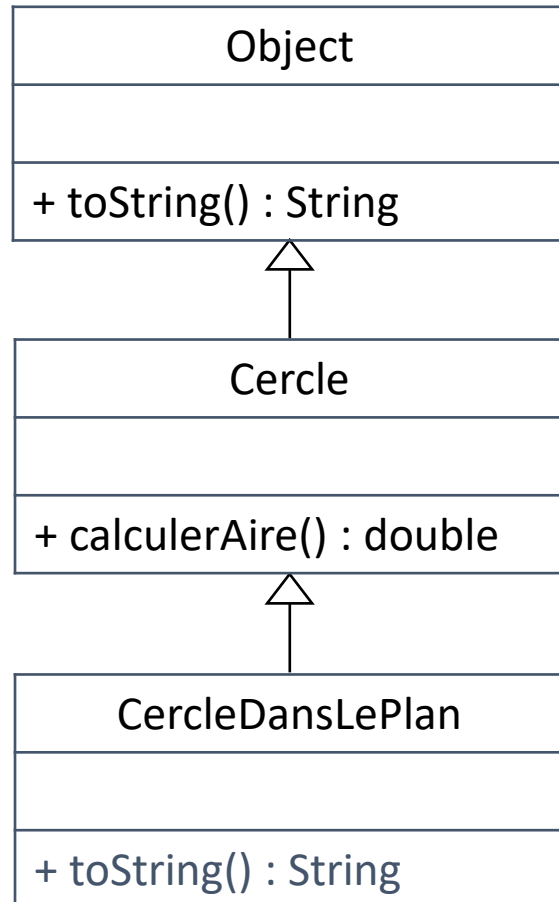
```
c.calculerAire();
```

✓ toString() existe dans Cercle. Utilisation de toString() le plus proche de CercleDansLePlan, c-à-d toString() de la classe CercleDansLePlan

✓ calculerAire() existe dans Cercle. Utilisation de calculerAire() le plus proche de CercleDansLePlan, c-à-d calculerAire() de la classe Cercle



Lien dynamique : exemple

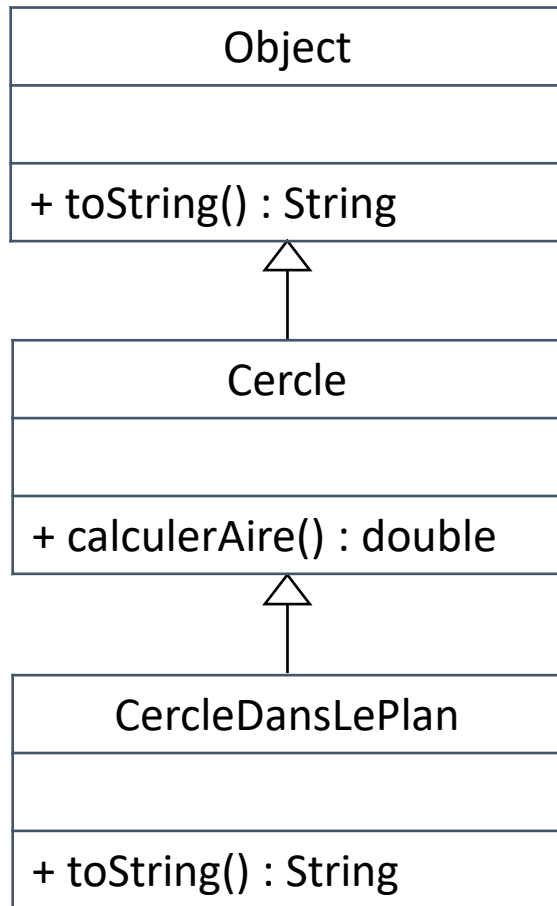


```
public class TestPolymorphisme{
    public static void main(String[] args){
        Point point = new Point(2,3);
        Cercle c = new CercleDansLePlan(1,point);
        System.out.println(c.toString());
    }
}
```

Quelle méthode `toString()` va être invoquée ?
Celle de `Cercle` ou celle de `CercleDansLePlan` ?



Exemple de polymorphisme



Que va afficher
ce programme?

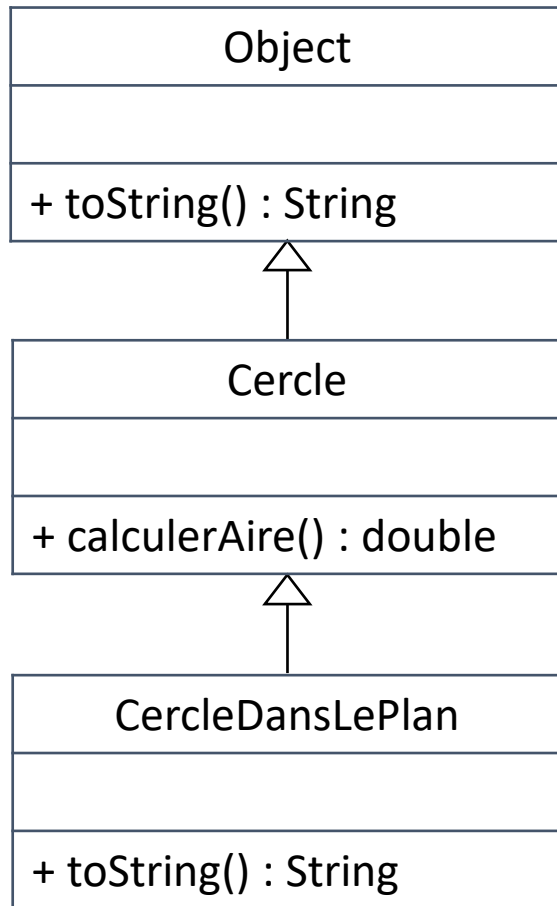
```
public class TestPolymorphisme{
    public static void main(String[] args){
        Point point = new Point(2,3);
        Cercle cercle = new CercleDansLePlan(1,point);
        System.out.println(cercle.toString());
        System.out.println("aire du cercle : " +
            cercle.calculerAire());
    }
}
```

C'est la méthode `toString()`
de la classe
`CercleDansLePlan` qui sera
exécutée

C'est la méthode
`calculerAire()` de la classe
`Cercle` qui sera exécutée



Exemple de polymorphisme



Que va afficher
ce programme?

```
public class TestPolymorphisme{
    public static void main(String[] args){
        Point point = new Point(2,3);
        Object o = new CercleDansLePlan(1,point);
        System.out.println(o.toString());
        System.out.println("aire du cercle : " +
            o.calculerAire());
    }
}
```

C'est la méthode `toString()` de
la classe `CercleDansLePlan`
qui sera exécutée

La méthode `calculerAire()`
n'existe pas pour `Object` !



Des questions

